

KrysukEngine

Kryptonim projektu: **Krysuk**

Dokumentacja projektu

Lekki, przenośny silnik gier 3D z architekturą ECS

Karyna Rudenko – team leader
Viktor Zasimovych – członek zespołu

23 czerwca 2026

Nazwa kodowa (kryptonim) projektu: Krysuk

Pełna nazwa przedsięwzięcia projektowego: KrysukEngine – lekki, przenośny silnik gier komputerowych 3D z wbudowanym renderem oraz architekturą ECS (Entity-Component-System)

[Repozytorium GitHub](#)

[Trello](#)

Spis treści

1	Wprowadzenie	4
1.1	Skład zespołu	4
1.2	Repozytorium i narzędzia	4
2	Wizja aplikacji	4
2.1	Jak nazywa się produkt?	4
2.2	Jakiej klasy jest produkt?	4
2.3	Dla kogo jest przeznaczony?	4
2.4	Jakie potrzeby zaspokaja?	4
2.5	Jakie korzyści przynosi, za które klient zechce zapłacić?	4
2.6	Jakie są alternatywy?	5
3	Karta projektu (Project Charter)	5
3.1	Temat projektu	5
3.2	Wizja systemu	5
3.2.1	Główne korzyści	5
3.3	Zakres usług	5
3.3.1	Ograniczenia – czego nie ma w projekcie	6
3.4	Architektura logiczna	6
3.4.1	Opis elementów	6
3.4.2	Komunikacja	6
3.5	Główne funkcje – epiki i historyjki użytkownika	7
4	Kryteria akceptacji i scenariusze testowe	7
4.1	Epik 1: Podstawowy silnik ECS	7
4.1.1	H1: Tworzenie encji i dodawanie komponentów	7
4.1.2	H2: Definiowanie własnych systemów	8
4.1.3	H3: Cykl życia systemów	9
5	Projekt interfejsu użytkownika	9
5.1	Diagram aktywności	9
5.2	Prototyp interfejsu Low-Fidelity	11
5.2.1	Toolbar (pasek narzędzi)	12
5.2.2	Hierarchy tab (hierarchia sceny)	12
5.2.3	Scene view (widok sceny)	12
5.2.4	Project window (okno projektu)	12
5.2.5	Inspector tab (inspektor)	13
6	Diagramy sekwencji	13
6.1	Start aplikacji i wczytanie sceny	13
6.2	Główna pętla	14
6.3	Pipeline renderowania pojedynczej ramki	15
7	Diagram klas	17

8	Podsumowania sprintów	17
8.1	Sprint 1 – 14 kwietnia 2026	17
8.1.1	Co zostało zrobione	17
8.1.2	Dalszy plan	18
8.2	Sprint 2 – 12 maja 2026	18
8.2.1	Co zostało zrobione	18
8.2.2	Dalszy plan	19
8.3	Sprint 3 – 3 czerwca 2026	20
8.3.1	Co zostało zrobione	20
8.3.2	Dalszy plan	20
8.4	Sprint 4 – 23 czerwca 2026	20
8.4.1	Co zostało zrobione	20
8.4.2	Testy jednostkowe	21
8.4.3	Dalszy plan	25
9	Raport końcowy projektu	25
9.1	Podsumowanie realizacji	25
9.2	Zrealizowany zakres funkcjonalny	25
9.3	Testy	26
9.4	Zgodność z zakresem projektu	26
9.5	Wnioski	26
9.6	Zalecenia na przyszłość	26

1 Wprowadzenie

Niniejszy dokument stanowi kompletną dokumentację projektu **KrysukEngine** (kryptonim: *Krysuk*), realizowanego w ramach przedmiotu Inżynieria Oprogramowania. Dokument obejmuje wszystkie etapy tworzenia projektu – od wizji aplikacji, przez kartę projektu i architekturę logiczną, aż po podsumowania kolejnych sprintów wraz z finalnym raportem zamykającym prace zespołu.

1.1 Skład zespołu

- **Karyna Rudenko** – team leader
- **Viktor Zasimovych** – członek zespołu

1.2 Repozytorium i narzędzia

- Repozytorium GitHub: github.com/karRudenko/KrysukEngine
- Tablica Trello: trello.com/b/GS1CTPHB/projekt-io

2 Wizja aplikacji

2.1 Jak nazywa się produkt?

Krysuk – nazwa handlowa produktu. **KrysukEngine** – kodowa nazwa projektu.

2.2 Jakiej klasy jest produkt?

Aplikacja desktopowa.

2.3 Dla kogo jest przeznaczony?

Dla deweloperów gier komputerowych.

2.4 Jakie potrzeby zaspokaja?

W przypadku deweloperów zaspokaja potrzebę tworzenia gier w oparciu o obiektowy silnik z paradygmatem ECS. Dostarcza wbudowany render oraz narzędzia ułatwiające integrację zasobów. Zapewnia przenośność między platformami (Linux, Windows). W przypadku gracza gwarantuje wysoką wydajność oraz umożliwia granie na różnych platformach.

2.5 Jakie korzyści przynosi, za które klient zechce zapłacić?

- Oszczędność czasu programistów – brak potrzeby tworzenia silnika od podstaw.
- Niższe koszty produkcji i utrzymania projektu.
- System ECS pozwala na elastyczny i niezależny rozwój komponentów oraz łatwe rozszerzanie funkcjonalności.

- Zintegrowany render skraca czas implementacji grafiki.
- Wsparcie popularnych formatów modeli 3D i tekstur przyspiesza pracę z assetami.
- Łatwiejsze utrzymanie kodu dzięki modularnej architekturze.
- Możliwość szybkiego portowania gry na różne platformy.

2.6 Jakie są alternatywy?

Tworzenie własnego silnika od podstaw albo korzystanie z alternatyw dostępnych na rynku (Unity, Unreal Engine, Godot itp.).

3 Karta projektu (Project Charter)

3.1 Temat projektu

Lekki, przenośny silnik gier 3D z architekturą ECS.

3.2 Wizja systemu

KrysukEngine to aplikacja desktopowa przeznaczona dla deweloperów gier komputerowych. System umożliwia tworzenie gier w oparciu o obiektowy silnik z paradygmatem ECS (Entity-Component-System). Dostarcza wbudowany render oraz narzędzia ułatwiające integrację zasobów. Zapewnia przenośność między platformami (Linux, Windows). Dla graczy gwarantuje wysoką wydajność i umożliwia granie na różnych platformach.

3.2.1 Główne korzyści

- Oszczędność czasu programistów – brak potrzeby tworzenia silnika od podstaw.
- Niższe koszty produkcji i utrzymania projektu.
- System ECS pozwala na elastyczny i niezależny rozwój komponentów.
- Zintegrowany render skraca czas implementacji grafiki.
- Wsparcie popularnych formatów modeli 3D i tekstur.
- Łatwiejsze utrzymanie kodu dzięki modularnej architekturze.
- Możliwość szybkiego portowania gry na różne platformy.

3.3 Zakres usług

W ramach projektu zostało zrealizowane:

- Implementacja podstawowego silnika ECS (Entity-Component-System).
- Wbudowany renderer 3D z obsługą podstawowych shaderów.
- Obsługa tekstur (PNG, JPG).

- System wejścia (klawiatura, mysz).
- Aplikacja desktopowa działająca na Windows i Linux.

3.3.1 Ograniczenia – czego nie ma w projekcie

- Edytor wizualny (scene editor) w wersji 1.0.
- Zaawansowany system fizyki (detekcja kolizji jest ograniczona).
- Silnik sieciowy (multiplayer).
- Wsparcie dla konsol (PlayStation, Xbox, Switch).
- Dźwięk i zaawansowane audio.
- Animacje szkieletowe.
- Silnik AI / ścieżkowy.

3.4 Architektura logiczna

System opiera się na modularnej architekturze, która umożliwia niezależny rozwój komponentów.

3.4.1 Opis elementów

- **Aplikacja desktopowa** – warstwa zarządzająca cyklem życia aplikacji (okno, pętla główna).
- **ECS Core** – serce silnika: zarządza encjami, komponentami i systemami.
- **Renderer 3D** – odpowiedzialny za rysowanie obiektów 3D na ekranie.
- **Manager assetów** – ładuje i przechowuje modele, tekstury, shadery.
- **System wejścia** – przetwarza zdarzenia z klawiatury i myszy.

3.4.2 Komunikacja

- Pętla główna aplikacji wywołuje kolejno: aktualizację ECS, przetworzenie wejścia, a następnie rendering.
- ECS Core modyfikuje dane komponentów, które są odczytywane przez renderer.
- System wejścia modyfikuje komponenty (np. pozycja kamery, ruch encji).
- Manager assetów dostarcza zasoby do renderera na żądanie.

3.5 Główne funkcje – epiki i historyjki użytkownika

Epik 1: Podstawowy silnik ECS

- H1: Jako deweloper chcę tworzyć encje i dodawać do nich komponenty, aby móc budować obiekty gry.
- H2: Jako deweloper chcę definiować własne systemy, które przetwarzają encje z określonymi komponentami.
- H3: Jako deweloper chcę, aby silnik automatycznie zarządzał cyklem życia systemów (init, update, destroy).

Epik 2: Renderer 3D

- H4: Jako deweloper chcę rysować modele 3D na ekranie z obsługą podstawowych shaderów.
- H5: Jako deweloper chcę ładować tekstury i nakładać je na modele.
- H6: Jako deweloper chcę sterować kamerą (obracanie, przybliżanie), aby testować scenę z różnych perspektyw.

Epik 3: Obsługa assetów

- H7: Jako deweloper chcę ładować modele 3D z plików OBJ, aby używać własnych zasobów.
- H8: Jako deweloper chcę ładować tekstury z plików PNG/JPG.

Epik 4: Przenośność między platformami

- H9: Jako deweloper chcę uruchomić tę samą grę na Windows i Linux bez zmian w kodzie.
- H10: Jako gracz chcę grać w gry stworzone w KrysukEngine na moim systemie operacyjnym (Windows/Linux).

4 Kryteria akceptacji i scenariusze testowe

W ramach projektu opracowano kompleksowe kryteria akceptacji dla wszystkich 10 historyjek użytkownika, pogrupowanych w 4 epiki. Dla każdej historyjki zdefiniowano szczegółowe scenariusze testowe w formacie GWT (Given-When-Then).

4.1 Epik 1: Podstawowy silnik ECS

4.1.1 H1: Tworzenie encji i dodawanie komponentów

Kryteria akceptacji:

- System umożliwia utworzenie nowej, unikalnej encji (Entity) w ramach ECS Core.
- Do istniejącej encji można dodać jeden lub więcej komponentów (np. Transform, Mesh).

- Każda encja może posiadać maksymalnie jeden komponent danego typu (brak duplikatów).
- Próba dodania komponentu do nieistniejącej encji zwraca błąd/wyjątek.
- Dane komponentu są odczytywalne natychmiast po jego dodaniu do encji.

Scenariusz testowy 1.1 – Utworzenie encji i dodanie komponentu

Given system ECS Core jest zainicjalizowany i nie zawiera żadnych encji
When developer wywołuje funkcję utworzenia nowej encji
Then system zwraca unikalny identyfikator nowej encji
And developer dodaje do tej encji komponent Transform
Then komponent Transform jest powiązany z encją i dostępny do odczytu

Scenariusz testowy 1.2 – Próba dodania duplikatu komponentu

Given encja posiada już przypisany komponent Transform
When developer próbuje dodać do tej samej encji kolejny komponent typu Transform
Then system odrzuca operację i zwraca informację o konflikcie typu komponentu

Scenariusz testowy 1.3 – Dodanie komponentu do nieistniejącej encji

Given identyfikator encji nie istnieje w systemie (np. została usunięta)
When developer próbuje dodać komponent do tego identyfikatora
Then system zgłasza błąd i nie modyfikuje stanu wewnętrznego ECS

4.1.2 H2: Definiowanie własnych systemów

Kryteria akceptacji:

- Deweloper może zarejestrować własny system, deklarując zestaw komponentów wymaganych do przetwarzania.
- System podczas każdej aktualizacji otrzymuje wyłącznie encje posiadające zadeklarowany zestaw komponentów.
- Możliwe jest zarejestrowanie wielu systemów działających niezależnie od siebie.
- Kolejność wykonywania systemów jest deterministyczna.
- Encje nieposiadające wymaganych komponentów są automatycznie pomijane przez system.

Scenariusz testowy 2.1 – Filtrowanie encji przez system

Given zarejestrowano system wymagający komponentów Transform i Velocity
w świecie istnieją trzy encje: dwie z oboma komponentami, jedna tylko z Transformem
When wywoływana jest aktualizacja (update) systemu
Then system przetwarza wyłącznie dwie encje posiadające oba wymagane komponenty

4.1.3 H3: Cykl życia systemów

Kryteria akceptacji:

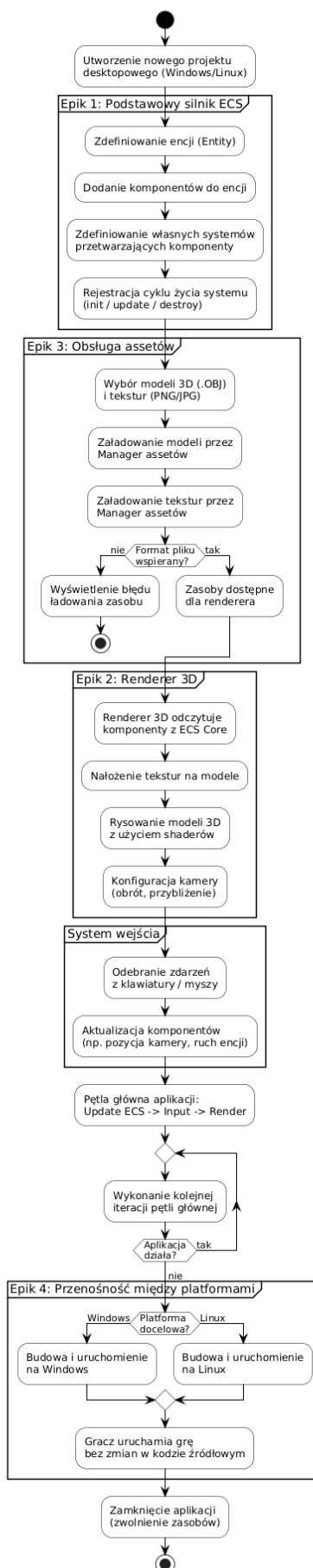
- Metoda `init()` systemu jest wywoływana automatycznie jednokrotnie, przed pierwszym wywołaniem `update()`.
- Metoda `update()` jest wywoływana w każdej iteracji pętli głównej aplikacji.
- Metoda `destroy()` jest wywoływana automatycznie przy zamknięciu aplikacji.
- Kolejność wywołań: `init` $\rightarrow$ `(update)*` $\rightarrow$ `destroy`.
- Błąd w `init()` zatrzymuje rejestrację danego systemu i jest logowany.

5 Projekt interfejsu użytkownika

5.1 Diagram aktywności

Diagram aktywności przedstawia przepływ pracy dewelopera z silnikiem KrysukEngine – od utworzenia nowego projektu, przez definiowanie encji i systemów ECS, ładowanie assetów, konfigurację renderera i kamery, aż po budowanie i uruchomienie gry na docelowych platformach.

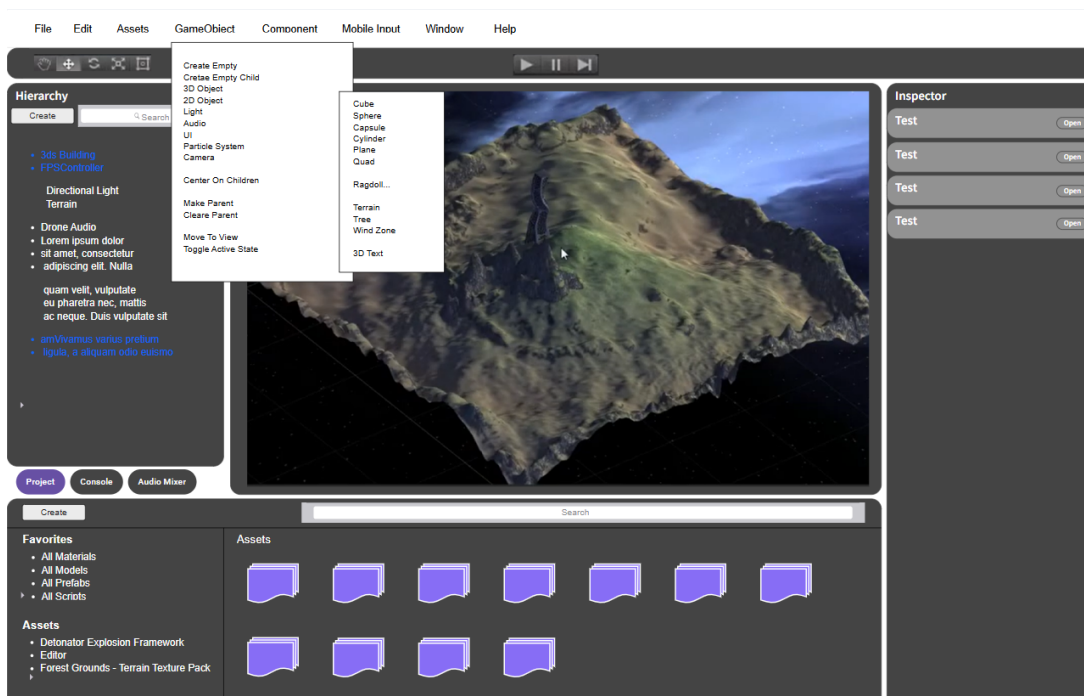
**Diagram aktywności - Przeływ pracy dewelopera z KrysukEngine
(Tworzenie i uruchomienie: ECS, Renderer, Assety, Wejście, Przeność)**



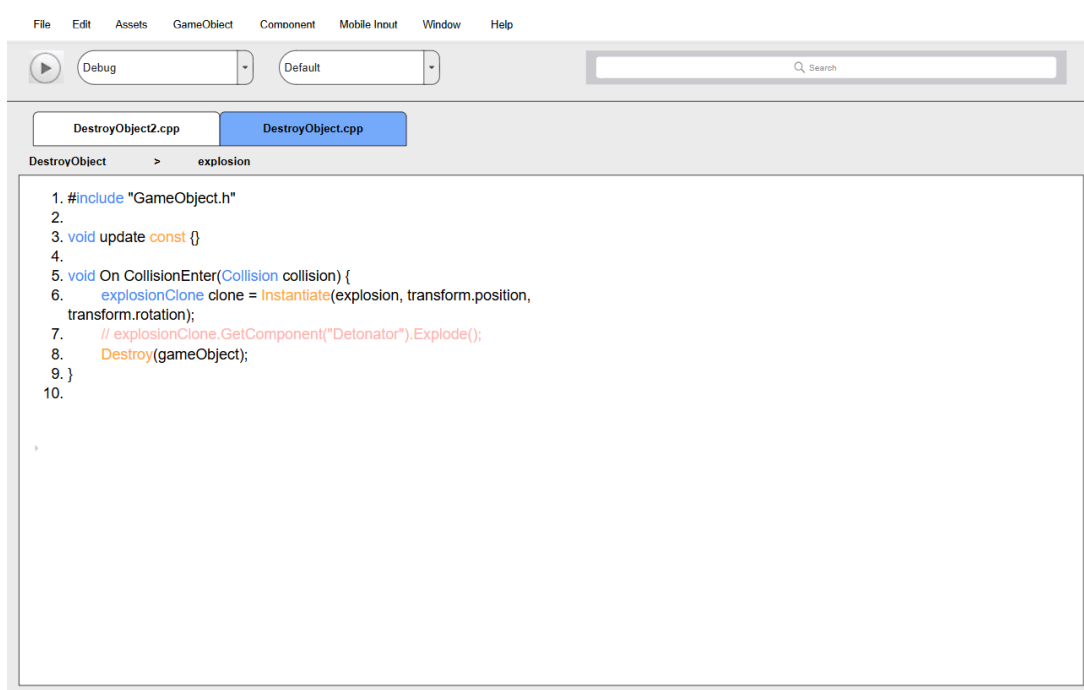
Rysunek 1: Diagram aktywności – przepływ pracy dewelopera z KrysukEngine

5.2 Prototyp interfejsu Low-Fidelity

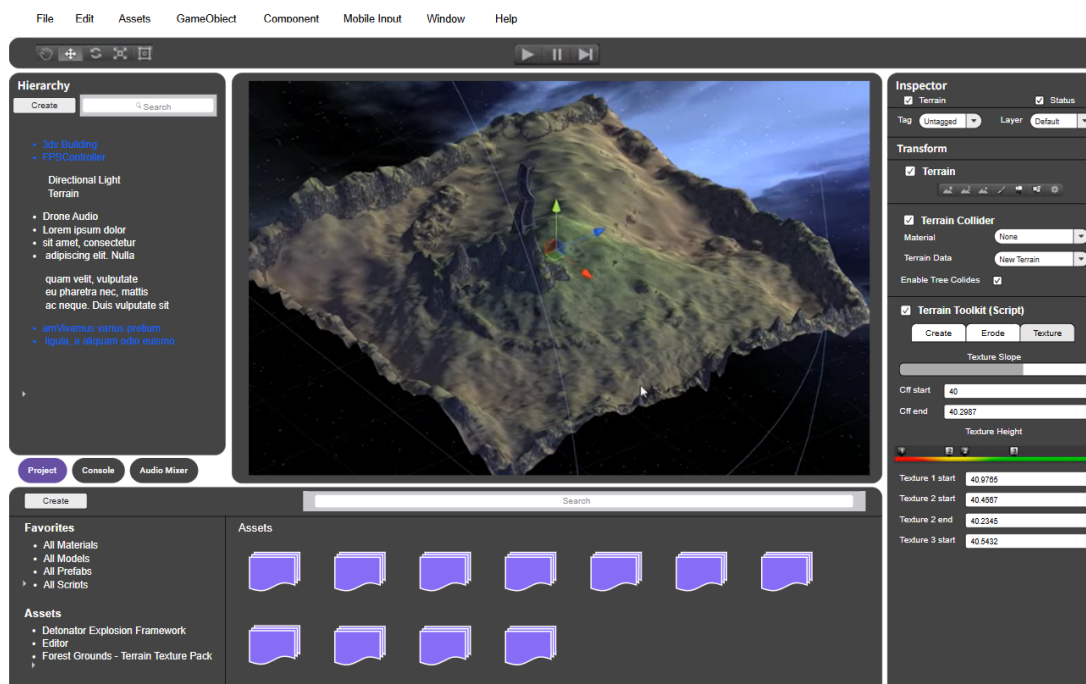
Poniżej przedstawiono opis makiety niskiej wierności (Lo-Fi) planowanego edytora sceny, ilustrującej docelowy układ interfejsu narzędzia wspierającego pracę z silnikiem oraz prototyp UI.



Rysunek 2: Prototyp głównego edytora – projektowanie sceny



Rysunek 3: Prototyp kodowego edytora – pisanie skryptów



Rysunek 4: Prototyp głównego edytora – zaawansowane narzędzie modyfikacji obiektów

5.2.1 Toolbar (pasek narzędzi)

- Górny pasek zawiera narzędzia transformacji: zaznaczanie, przesuwanie, obracanie, skalowanie, prostokąt.
- Przyciski Play / Pause / Step w centrum umożliwiają uruchamianie podglądu gry.

5.2.2 Hierarchy tab (hierarchia sceny)

- Panel po lewej stronie wyświetla drzewo obiektów aktywnej sceny.
- Obiekty można grupować jako rodzic-dziecko (zagnieżdżanie).
- Przycisk "Create" umożliwia dodanie nowego obiektu do sceny.
- Pole wyszukiwania filtruje obiekty po nazwie.

5.2.3 Scene view (widok sceny)

- Główny obszar roboczy z siatką 3D/2D – tutaj odbywa się wizualna edycja.
- Pasek trybu: Shaded/Wireframe, 2D/3D, oświetlenie, audio, nakładki.
- Obiekty można zaznaczać, przesuwać i skalować bezpośrednio w widoku.

5.2.4 Project window (okno projektu)

- Dolny panel pokazuje strukturę plików projektu (zasoby / assets).
- Lewy panel: drzewo folderów (Scenes, Scripts, Prefabs, Materials, Audio).

- Prawy panel: ikony zasobów w wybranym folderze.
- Zakładka Console wyświetla logi, błędy i ostrzeżenia ze skryptów.

5.2.5 Inspector tab (inspektor)

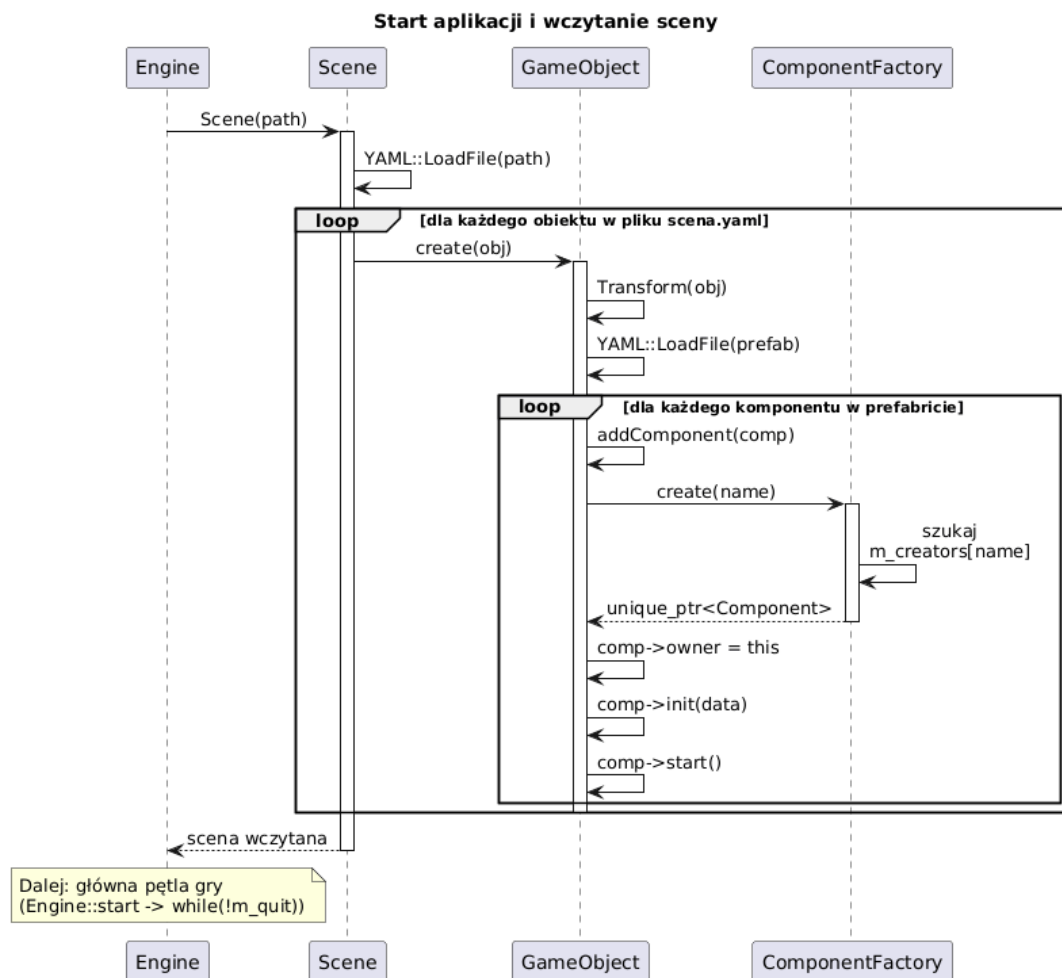
- Prawy panel wyświetla komponenty przypisane do zaznaczonego obiektu.
- Każdy komponent (Transform, Renderer, Rigidbody, Skrypt) można rozwijać/zwijać.
- Pola wartości (float, bool, wektor, maska) można edytować bezpośrednio.

6 Diagramy sekwencji

W ramach projektu opracowano trzy diagramy sekwencji UML obrazujące kluczowe procesy systemu.

6.1 Start aplikacji i wczytanie sceny

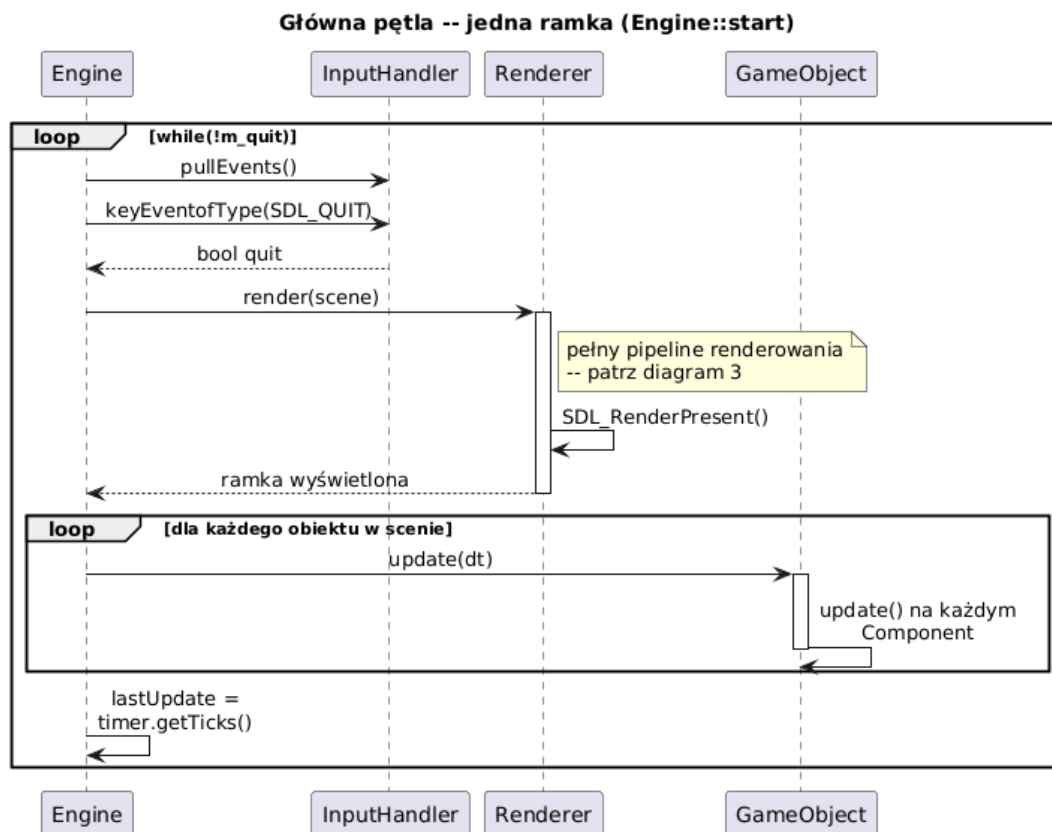
Diagram przedstawia proces inicjalizacji silnika, wczytywanie pliku sceny w formacie YAML, tworzenie obiektów gry na podstawie prefabrykatów oraz dodawanie komponentów z wykorzystaniem wzorca Fabryki. Sekwencja kończy się przejściem do głównej pętli programu.



Rysunek 5: Diagram sekwencji – start aplikacji i wczytanie sceny

6.2 Główna pętla

Diagram obrazuje działanie pętli głównej (`Engine::start`) działającej do momentu ustawienia flagi `m_quit`. W każdej iteracji pętli wykonywane są: pobieranie zdarzeń wejściowych (`pullEvents`), obsługa zdarzeń (np. `SDL_QUIT`) oraz renderowanie sceny. Dla każdego obiektu w scenie wywoływana jest aktualizacja (`update(dt)`) z uwzględnieniem czasu od ostatniej klatki.

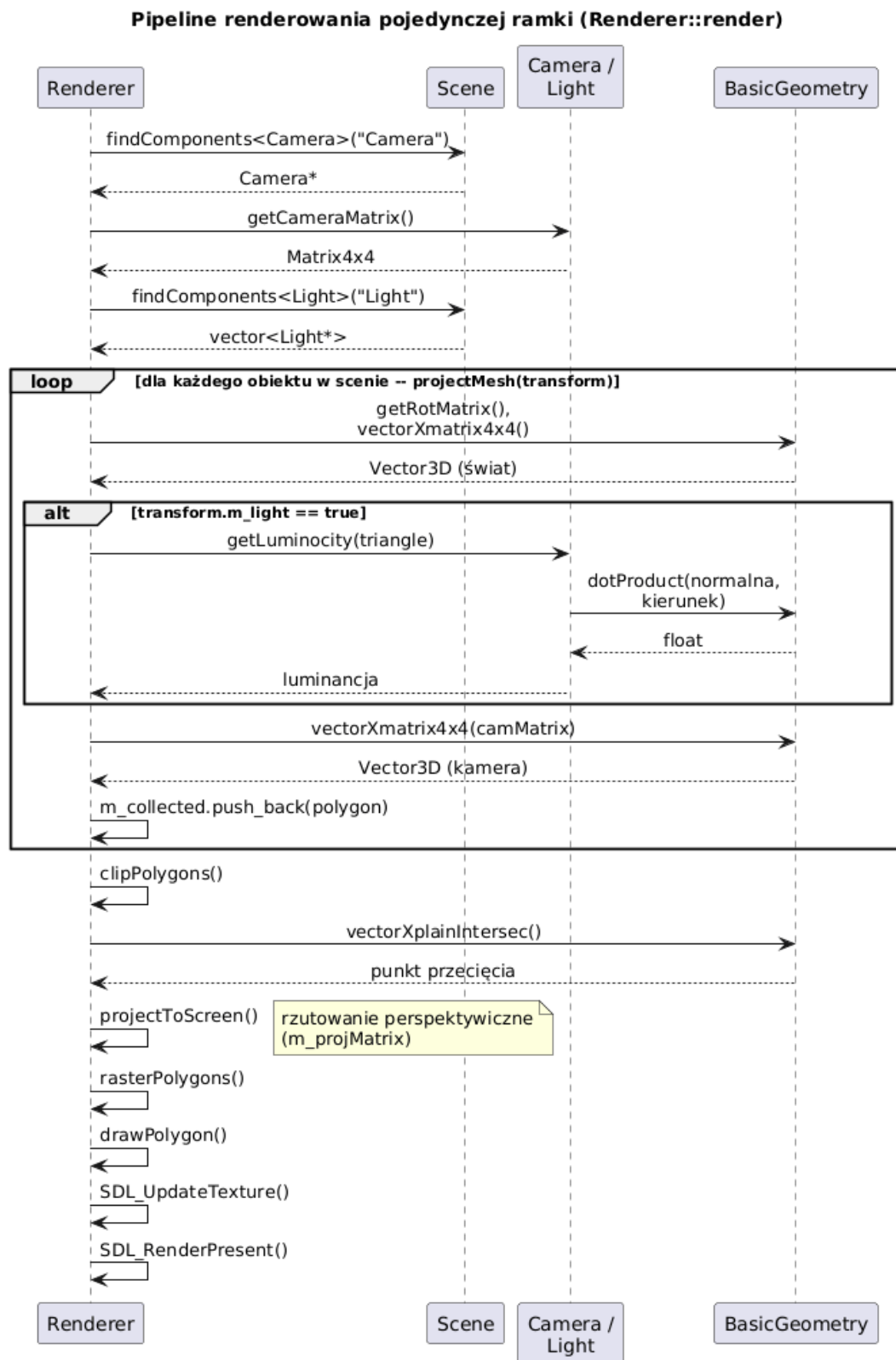


Rysunek 6: Diagram sekwencji – główna pętla (jedna ramka)

6.3 Pipeline renderowania pojedynczej ramki

Diagram przedstawia szczegółowy proces renderowania (`Renderer::render`). Sekwencja obejmuje:

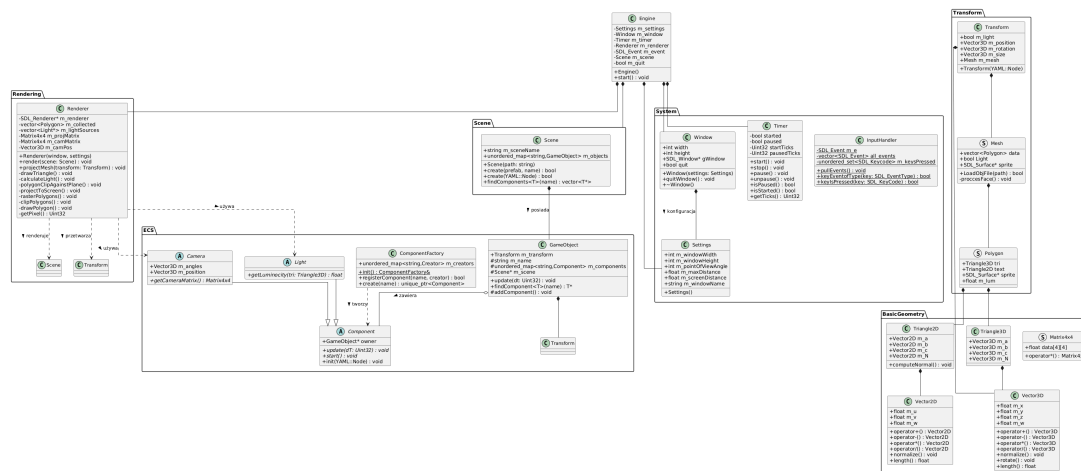
- pobranie komponentów kamery i światła ze sceny,
- obliczenie macierzy kamery i transformacji obiektów,
- dla każdego obiektu w scenie: transformację wierzchołków, obliczenie luminancji (z uwzględnieniem światła i normalnych), rzutowanie perspektywiczne,
- przycinanie wielokątów do frustum kamery (`clipPolygons`),
- rasteryzację i rysowanie wielokątów na ekranie (`rasterPolygons`, `drawPolygon`),
- aktualizację tekstury i prezentację ramki za pomocą SDL.



Rysunek 7: Diagram sekwencji – pipeline renderowania pojedynczej ramki

7 Diagram klas

Poniżej przedstawiono diagram klas opisujący strukturę zaimplementowanego systemu, obejmujący moduły: Engine, Rendering, ECS, Scene, Transform, System oraz BasicGeometry.



Rysunek 8: Diagram klas systemu KrysukEngine

Główne moduły architektury:

- **Engine** – klasa centralna zarządzająca cyklem życia aplikacji (ustawienia, okno, timer, renderer, scena).
- **Rendering** – moduł **Renderer** odpowiedzialny za rzutowanie, oświetlanie, rasteryzację i przycinanie wielokątów.
- **ECS** – moduł zawierający **GameObject**, **Component**, **ComponentFactory** oraz klasy bazowe **Camera** i **Light**.
- **Scene** – przechowuje kolekcję obiektów gry (**GameObject**) oraz umożliwia ich tworzenie na podstawie plików YAML.
- **Transform** – przechowuje pozycję, rotację, skalę oraz siatkę (**Mesh**) obiektu.
- **System** – moduł obsługujący okno (**Window**), czas (**Timer**), wejście (**InputHandler**) oraz konfigurację (**Settings**).
- **BasicGeometry** – biblioteka matematyczna: wektory 2D/3D, trójkąty oraz macierze 4x4.

8 Podsumowania sprintów

8.1 Sprint 1 – 14 kwietnia 2026

8.1.1 Co zostało zrobione

- **Identyfikacja i zebranie wymagań użytkownika** (Karyna) – przeprowadzono analizę w celu wyłonienia kompletnej listy wymagań funkcjonalnych i нефункциональных.

- **Opis wymagań w postaci historyjek użytkownika** (Viktor) – przekształcono zebrane wymagania na zbiór historyjek użytkownika według wzorca: "Jako [rola], chcę [cel], aby [korzystać]".
- **Rozwinięcie historyjek w kryteria akceptacji** (Viktor) – dla każdej historyjki zdefiniowano wstępne kryteria akceptacji oraz podstawowe przypadki użycia.
- **Identyfikacja scenariuszy alternatywnych i wyjątków** (Karyna) – rozszerzono przypadki użycia o scenariusze alternatywne.

Wektory (Karyna)

- Klasa wektorów 4D.
- Arytmetyka wektorów (operacje ze skalarami).
- Klasa trójkątów Triangle3D, Triangle2D.

Macierze (Viktor)

- Klasa macierzy 4x4.
- Arytmetyka macierzy.

Konfiguracja OpenGL (Viktor)

- Konfiguracja SDL, tworzenie API dla okna.
- Konfiguracja OpenGL, podłączenie do SDL.

8.1.2 Dalszy plan

- Kompletne kryteria akceptacji i scenariusze testowe w formacie GWT.
- Projekt flow użytkownika (Diagram aktywności UML).
- Transform – dodanie Mesh, położenia i obrotu.
- GameObject i Scene – integracja Transform, klasa Scene, rozszerzenie Renderer.

8.2 Sprint 2 – 12 maja 2026

8.2.1 Co zostało zrobione

- **Kompletne kryteria akceptacji i scenariusze testowe w formacie GWT** (Karyna) – dla każdej historyjki użytkownika opracowano szczegółowe scenariusze testowe przy użyciu wzorca GWT (Given-When-Then). Zidentyfikowano przypadki brzegowe.
- **Projekt flow użytkownika – diagramy aktywności** (Karyna) – stworzono diagramy aktywności UML dla kluczowych ścieżek użytkownika.
- **Wizualizacja interfejsu – prototyp low-fidelity** (Karyna, Viktor) – przygotowano interaktywny prototyp interfejsu w wersji low-fidelity zawierający szkice wszystkich kluczowych ekranów aplikacji.

- **Przygotowanie środowiska do testów użyteczności** (Viktor) – opracowano scenariusze testów użyteczności oraz materiały dla moderatora.

Wektory (Karyna)

- Klasa wektorów 4D.
- Arytmetyka wektorów (operacje ze skalarami).
- Klasa trójkątów `Triangle3D`, `Triangle2D`.

Macierze (Viktor)

- Klasa macierzy 4x4.
- Arytmetyka macierzy.

Konfiguracja OpenGL (Viktor)

- Konfiguracja SDL, tworzenie API dla okna.
- Konfiguracja OpenGL, podłączenie do SDL.

Mesh (Viktor)

- Struktura `Polygon`, zawierająca dane UV i modelu.
- Klasa `Mesh`, przechowująca polygony.
- Zmodyfikowanie `Render`, dodanie możliwości wczytywania plików `.obj` (bez tekstur).

Transform (Karyna)

- Dodanie `Mesh`.
- Dodanie położenia i obrotu w globalnym układzie współrzędnych.

GameObject i Scene (Karyna)

- Integracja `Transform` do `GameObject`.
- Klasa `Scene`, przechowująca `GameObject`.
- Rozszerzenie klasy `Renderer` do przejmowania `Scene`.

8.2.2 Dalszy plan

- Badania interfejsu w gronie użytkowników – przeprowadzenie sesji testów użyteczności.
- Architektura systemu – diagramy klas i pakietów.
- Diagramy sekwencji – obrazujące przepływ żądań pomiędzy częściami systemu.
- Konfiguracja projektu – utworzenie folderu `lib`, konfiguracja `CMake`, konfiguracja `pkgconfig`.
- Kamera – zaimplementowanie klasy `Camera`, dodanie do renderowania.
- Oświetlenie (`Light`) – tworzenie klasy `Light`, integracja do projektu.

8.3 Sprint 3 – 3 czerwca 2026

8.3.1 Co zostało zrobione

- **Diagramy klas** (Karyna) – stworzono szczegółowe diagramy klas w UML dla warstwy logiki, obejmujące główne encje, relacje pomiędzy klasami oraz wzorce projektowe.
- **Diagramy sekwencji** (Viktor) – przygotowano szczegółowe diagramy sekwencji UML dla kluczowych procesów systemu:
 - Start aplikacji i wczytanie sceny – inicjalizacja silnika, wczytywanie pliku YAML, tworzenie obiektów gry.
 - Główna pętla – działanie pętli głównej z pobieraniem zdarzeń i renderowaniem.
 - Pipeline renderowania pojedynczej ramki – transformacja wierzchołków, obliczanie luminancji, rzutowanie perspektywiczne, przycinanie, rasteryzacja.

Konfiguracja projektu (Karyna)

- Utworzenie folderu `lib`.
- Konfiguracja CMake.
- Konfiguracja pkgconfig.

Kamera (Viktor)

- Zaimplementowanie klasy `Camera`.
- Dodanie kamery do renderowania.

Oświetlenie (Light) (Karyna)

- Tworzenie klasy `Light`.
- Integrowanie klasy `Light` do projektu.

8.3.2 Dalszy plan

- Implementacja projektu (backend wraz z testami jednostkowymi).
- Przygotowanie prezentacji projektu.
- Skompletowanie dokumentacji końcowej.

8.4 Sprint 4 – 23 czerwca 2026

8.4.1 Co zostało zrobione

Konfiguracja budowania (CMake)

- Rozbudowanie pliku `CMakeLists.txt` o automatyczne wykrywanie plików źródłowych dla katalogów `src` oraz `util`.

- Dodanie mechanizmu automatycznego generowania pliku rejestru komponentów na etapie konfiguracji CMake.
- Skonfigurowanie odrębnego celu budowania UTIL dla narzędzi pomocniczych (folder `util`), niezależnego od głównego pliku wykonywalnego `program`.
- Podłączenie bibliotek zewnętrznych: `SDL2`, `SDL2main`, `SDL2_image`, `SDL2_ttf`, `SDL2_mixer` oraz `yaml-cpp`.

Narzędzie generatora komponentów (`util/CompGenerator.cpp`)

- Implementacja samodzielnego programu konsolowego `CompGenerator`, który na podstawie nazwy istniejącej klasy bazowej (np. `Camera`, `Light`) oraz nazwy nowego komponentu generuje automatycznie:
 - plik nagłówkowy nowej klasy dziedziczącej po klasie bazowej, z zachowaniem sekcji `public/private` oraz nadpisaniem metod oznaczonych jako `virtual` (z dodanym specyfikatorem `override`),
 - odpowiadający plik `.cpp` zawierający szkielet implementacji oraz funkcję rejestrującą komponent w `ComponentFactory`.
- Mechanizm parsowania pliku nagłówkowego klasy bazowej i podstawowa transformacja tekstu w celu wygenerowania poprawnej składni C++.

8.4.2 Testy jednostkowe

W ramach ostatniego sprintu opracowano zestaw 61 automatycznych testów jednostkowych przy użyciu frameworka **Google Test**. Wynik uruchomienia: **61 testów**.

Sposób uruchomienia testów

```
# Kompilacja (Linux/g++, wymagana biblioteka libgtest-dev)
g++ -std=c++17 -I include -I yaml-cpp-0.8.0/include \
    -I /usr/src/gtest/include \
    tests/test_engine.cpp \
    src/BasicGeometry.cpp src/ComponentFactory.cpp src/Component.cpp \
    /usr/src/gtest/build/lib/libgtest.a \
    /usr/src/gtest/build/lib/libgtest_main.a \
    -lpthread -o tests/run_tests

# Uruchomienie
./tests/run_tests
```

Vector2D – 11 testów

- `Konstruktor_ZapisujePola` – poprawne przypisanie pól `m_u`, `m_v`.
- `Dodawanie` – sumowanie składowych dwóch wektorów 2D.
- `OdejmowanieWektora` – odejmowanie składowych dwóch wektorów 2D.
- `OdejmowanieSkalarU` – odejmowanie skalaru od obu składowych.

- `MnozeniePrzezSkalar` – mnożenie składowych przez skalar.
- `DzieleniePrzezSkalar` – dzielenie składowych przez skalar.
- `Dlugosc_TrojkaPytag` – długość wektora (3,4) równa 5 (trójka pitagorejska).
- `Dlugosc_ZerowegoWektora` – długość wektora zerowego wynosi 0.
- `Normalize_DajeJednostkowy` – po normalizacji długość wynosi 1.
- `Normalize_NieZmieniaKierunkuDodatniego` – kierunek wektora pionowego (0,5) po normalizacji to (0,1).
- `Normalize_WektorZerowy_BezKraksy` – normalizacja wektora (0,0) nie rzuca wyjątku.

Vector3D – 12 testów

- `Konstruktor_ZapisujePola` – poprawne przypisanie `m_x`, `m_y`, `m_z`.
- `DomyslnyKonstruktor_DajeZera` – domyślny konstruktor inicjuje wektor zerowy.
- `Dodawanie, OdejmowanieWektora` – operacje składowe.
- `MnozeniePrzezSkalar, DzieleniePrzezSkalar` – skalowanie wektora.
- `Dlugosc_TrojkaPytag3D` – długość wektora (1,2,2) równa 3.
- `Dlugosc_WektoraJednostkowego` – długość (1,0,0) wynosi 1.
- `Normalize_DajeJednostkowy` – po normalizacji długość wynosi 1.
- `Normalize_KierunekX` – wektor (5,0,0) po normalizacji to (1,0,0).
- `Rotate_Zerowy_BezZmiany` – obrót o (0°,0°,0°) nie zmienia wektora.
- `Rotate_360StopniDajeOryginal` – obrót o 360° wokół osi Y zwraca oryginalny wektor (tolerancja 1e-3).

Funkcje geometryczne (`dotProduct`, `crossProduct`) – 8 testów

- `DotProduct_ProstopadleWektory` – iloczyn skalarny wektorów prostopadłych wynosi 0.
- `DotProduct_RownolegleWektory` – iloczyn skalarny równoległych wektorów $(2,0,0) \cdot (3,0,0) = 6$.
- `DotProduct_PrzeciwnieWektory` – iloczyn skalarny przeciwnych wektorów wynosi -1.
- `DotProduct_OgolnyPrzyklad` – $(1,2,3) \cdot (4,5,6) = 32$.
- `CrossProduct_DwaWektory_Prostopadlosc` – $x \times y = (0,0,1)$.
- `CrossProduct_TrzeciBazowy` – $y \times x = (0,0,-1)$.

- `CrossProduct_TrzyPunkty_NormalTriangla` – iloczyn wektorowy trójkąta w płaszczyźnie XY daje wektor normalny równoległy do osi Z.
- `CrossProduct_RownolegleDajaZerowy` – iloczyn wektorowy równoległych wektorów wynosi (0,0,0).

Matrix4x4 – 10 testów

- `GetIdentMatrix_MaJedynkiNaPrzekatnej` – macierz identyczności ma 1 na przekątnej, 0 poza nią.
- `MatrixXMatrix_IdentPrzezIdent_DaIdent` – $I \cdot I = I$.
- `MatrixXMatrix_IdentPrzezDowolna_DajeTaSama` – $I \cdot M = M$ dla dowolnej macierzy M.
- `GetTransMatrix_UstawiaTranslacje` – macierz translacji o (3,5,7) ma te wartości w ostatnim wierszu.
- `VectorXMatrix_IdentNieZmieniaWektora` – mnożenie wektora przez I nie zmienia go.
- `QuickInverse_IdentPrzezIdentDaIdent` – odwrotność macierzy I to I.
- `QuickInverse_OdwracaMacierzObrotu` – $R \cdot R^{-1} \approx I$ (tolerancja 1e-3).
- `GetRotMatrix_Zero_DaIdentycznosc` – macierz obrotu o (0°,0°,0°) jest macierzą identyczności.
- `GetRotMatrix_90Y_ObraCaOsX` – obrót wektora (1,0,0) o 90° wokół Y zmienia składowe x lub z.
- `OperatorMnozenia_Przemienosc_IdentPrzezIdent` – operator * macierzy: $I * I$ daje 1 na przekątnej.

Triangle3D – 3 testy

- `Konstruktor_ZapisujePunkty` – wierzchołki trójkąta są poprawnie przechowywane.
- `Normalna_ProstopadlaDoPlaszczyzny` – normalna trójkąta w XY jest prostopadła do jego krawędzi i ma długość 1.
- `KonstruktorZNormalna_UzywaJejPoBezposredniej` – konstruktor z podaną normalną normalizuje ją poprawnie.

Triangle2D – 4 testy

- `Konstruktor_ZapisujePunkty` – wierzchołki 2D są poprawnie przechowywane.
- `Normalna_JestZnormalizowana` – automatycznie obliczona normalna ma długość 1.
- `ComputeNormal_ProstopadlaDoKrawedzi` – normalna jest prostopadła do krawędzi AB trójkąta.
- `KonstruktorZNormalna_Normalizuje` – konstruktor normalizuje podaną normalną $(0,10) \rightarrow (0,1)$.

Przecięcie z płaszczyzną (`vectorXplainIntersec`) – 3 testy

- `SrodekOdcinka_NaPlaszczyznie` – punkt przecięcia ze środkiem odcinka leży na płaszczyźnie $y=0$, parametr $t=0.5$.
- `PrzeciecieWEkscentrycznym_PunkcieOdcinka` – dla płaszczyzny $x=3$ i odcinka $[0,6]$ punkt przecięcia leży w $x=3$.
- `Parametr_t_Rowny1_GdyKoniecNaPlanie` – gdy koniec odcinka leży na płaszczyźnie, parametr $t=1$.

`getPointAtMatrix` – 2 testy

- `MacierzNieJestZerowa` – wygenerowana macierz widoku kamery jest niezerowa.
- `PozycjaPrzechowywanaWOstatnimWierszu` – pozycja kamery $(3,5,7)$ jest zapisywana w ostatnim wierszu macierzy.

ComponentFactory (ECS) – 8 testów

- `Rejestracja_ZwracaTrueGdyNowaKlasa` – rejestracja nowego komponentu zwraca `true`.
- `Rejestracja_ZwracaFalseGdyDuplicat` – ponowna rejestracja tej samej nazwy zwraca `false`.
- `Create_ZnanaNazwa_ZwracaNieNullptr` – tworzenie zarejestrowanego komponentu zwraca niepusty wskaźnik.
- `Create_NieznanaKlasa_ZwracaNullptr` – tworzenie nieznanego komponentu zwraca `nullptr`.
- `Create_ZwracaPoprawnyTypKomponentu` – zwrócony wskaźnik można bezpiecznie rzutować na właściwy typ (`dynamic_cast`).
- `Create_DwaRozneKomponenty` – fabryka poprawnie rozróżnia i tworzy różne typy komponentów.
- `Create_KazdyCallZwracaNowaInstancje` – każde wywołanie `create()` tworzy odrębną instancję (różne adresy).
- `Rejestracja_WieluKomponentow_WszystkieDostepne` – po rejestracji 5 różnych komponentów każdy z nich można utworzyć.

8.4.3 Dalszy plan

Sprint zamykał zaplanowany zakres prac projektu zgodny z kartą projektu. Dalszy rozwój silnika mógłby obejmować m.in. testy integracyjne renderera, wsparcie dla tekstur w renderze oraz edytor sceny.

9 Raport końcowy projektu

9.1 Podsumowanie realizacji

Projekt KrysukEngine został zrealizowany w czterech sprintach, w okresie od 14 kwietnia 2026 do 23 czerwca 2026. W tym czasie zaimplementowano podstawową wersję lekkiego silnika gier 3D opartego na architekturze ECS, zgodnie z założeniami przyjętymi w karcie projektu.

9.2 Zrealizowany zakres funkcjonalny

- **Biblioteka matematyczna (BasicGeometry)** – klasy `Vector2D`, `Vector3D`, `Triangle2D`, `Triangle3D` oraz `Matrix4x4` wraz z pełną arytmetyką wektorową i macierzową.
- **Warstwa systemowa (System)** – klasy `Window` (konfiguracja okna SDL), `Timer` (zarządzanie czasem, pauza, FPS), `InputHandler` (statyczna obsługa zdarzeń klawiatury/myszy) oraz `Settings` (wczytywanie konfiguracji z pliku YAML).
- **Renderer 3D** – klasa `Renderer` realizująca rzutowanie wielokątów (`projectMesh`, `projectToScreen`), rasteryzację (`rasterPolygons`, `drawPolygon`, `drawTriangle`), przycinanie wielokątów względem płaszczyzn (`polygonClipAgainstPlane`, `clipPolygons`) oraz obliczanie oświetlenia (`calculateLight`) na podstawie zarejestrowanych źródeł światła.
- **Wczytywanie modeli** – klasa `Mesh` ze statyczną metodą `LoadObjFile`, umożliwiającą wczytywanie geometrii z plików `.obj`.
- **Transform** – klasa przechowująca pozycję, rotację, skalę oraz siatkę (`Mesh`) obiektu w globalnym układzie współrzędnych, konfigurowalna z węzła YAML.
- **Architektura ECS** – klasy `GameObject`, `Component` (klasa abstrakcyjna z cyklem życia `init/start/update`) oraz `ComponentFactory` realizująca wzorzec fabryki z rejestracją komponentów po nazwie i tworzeniem ich na podstawie definicji w plikach YAML.
- **Scena** – klasa `Scene` przechowująca kolekcję obiektów gry w mapie skojarzeniowej, z możliwością wczytania całej sceny z pliku YAML oraz wyszukiwania komponentów po typie i nazwie (`findComponents<T>`).
- **Kamera** – klasa bazowa `Camera` oraz jej konkretna implementacja `Camera3D`, obliczająca macierz widoku na podstawie pozycji i rotacji obiektu.
- **Oświetlenie** – klasa bazowa `Light` oraz dwie konkretne implementacje: `FlatLight` (światło kierunkowe) i `PointLight` (światło punktowe), obliczające luminancję powierzchni trójkątów.

- **Sterowanie** – komponent `rotation` obsługujący ruch i obrót obiektu z klawiatury, wykorzystujący statyczny `InputHandler`.
- **Generator komponentów** – samodzielne narzędzie `CompGenerator`, automatyzujące tworzenie szkieletów nowych komponentów ECS na podstawie istniejących klas bazowych.
- **Konfiguracja budowania** – w pełni zautomatyzowany system budowania na bazie CMake, z automatycznym wykrywaniem plików źródłowych oraz dynamicznym generowaniem rejestru komponentów silnika, integracją bibliotek SDL2 (SDL2, SDL2_image, SDL2_ttf, SDL2_mixer) oraz biblioteki `yaml-cpp` do parsowania plików konfiguracyjnych i scen.
- **Testy jednostkowe** – zestaw 61 testów automatycznych (Google Test) dla modułów `BasicGeometry` i `ComponentFactory`, pokrywający klasy `Vector2D`, `Vector3D`, `Matrix4x4`, `Triangle2D`, `Triangle3D` oraz wzorzec fabryki ECS.

9.3 Testy

W ramach projektu opracowano zestaw **61 automatycznych testów jednostkowych** (framework Google Test), które z sukcesem weryfikują poprawność biblioteki matematycznej (`BasicGeometry`) oraz systemu rejestracji i tworzenia komponentów ECS (`ComponentFactory`). Testy nie wymagają środowiska graficznego i mogą być uruchamiane niezależnie od SDL. Pokrycie testami obejmuje 9 grup testowych: `Vector2DTest` (11), `Vector3DTest` (12), `FunkcjeGeometriiTest` (8), `Matrix4x4Test` (10), `Triangle3DTest` (3), `Triangle2DTest` (4), `PrzeciecieZPlanem` (3), `PointAtMatrixTest` (2), `ComponentFactoryTest` (8).

9.4 Zgodność z zakresem projektu

Zrealizowany zakres funkcjonalny jest zgodny z założeniami przyjętymi w karcie projektu (sekcja 3): zaimplementowano podstawowy silnik ECS, wbudowany renderer 3D, wczytywanie modeli z plików OBJ oraz system wejścia z klawiatury. Zgodnie z założonymi ograniczeniami, w projekcie nie zaimplementowano edytora wizualnego, zaawansowanej fizyki, silnika sieciowego, wsparcia dla konsol, dźwięku, animacji szkieletowych ani silnika AI.

9.5 Wnioski

Projekt KrysukEngine zrealizował podstawowe założenia karty projektu w zakresie silnika 3D z architekturą ECS. Architektura modułowa (rozdzielenie warstw: geometrii, systemu, renderera, ECS i sceny) umożliwiła niezależny rozwój poszczególnych komponentów przez członków zespołu w trakcie kolejnych sprintów. Wprowadzenie generatora komponentów w ostatnim sprincie istotnie usprawniło proces dodawania nowych typów komponentów ECS, eliminując część powtarzalnej pracy programistycznej.

9.6 Zalecenia na przyszłość

- Rozszerzenie testów jednostkowych o testy integracyjne renderera, wymagające środowiska graficznego.

- Implementacja wsparcia dla tekstur w renderze.
- Rozwój edytora sceny jako narzędzia wspomagającego pracę dewelopera.
- Dodanie obsługi większej liczby formatów plików modeli i tekstur.
- Optymalizacja wydajnościowa renderera dla większych scen.

[Repozytorium GitHub](#)

[Trello](#)